Guía de estudio

Arquitectura de Redes I — PEI 1

Tema 1: Arquitecturas de red | Tema 2: Protocolos de la capa de aplicación

Qué entra en la PEI 1 (guía docente 2026/27)

La PEI 1 se hace hacia mitad de cuatrimestre, pesa el **30 %** y cubre los **Temas 1 y 2**, con teoría y problemas (cuestiones de desarrollo o test y ejercicios). Resultados de aprendizaje evaluados: RA1–RA8, RA10 y RA11.

Contenido oficial	Dónde está en esta guía
T1: elementos de red, protocolos, retardos, lógica de redes, OSI, TCP/IP	Sección 1 (RA1–RA6)
T2: aplicaciones distribuidas, cliente/servidor, HTTP, DNS, sockets	Sección 2 (RA7, RA8, RA10)
Cálculo de parámetros de eficiencia	Subsecciones 1.2, 1.4 y 1.8, ejercicios y hoja de fórmulas (RA6)
Exámenes de años anteriores (PEI 1, EE, PP)	Sección 4: problemas resueltos y respuestas de test; DNS y HTTP en 2.3 y 2.4

Formato visto en exámenes anteriores: test de 10 preguntas (5 puntos; acierto $+0{,}5$, fallo $-0{,}1$, blanco 0) y dos problemas (de 2 y 3 puntos). FTP, correo (SMTP, POP3, IMAP) y BitTorrent no figuran en los contenidos oficiales (solo web y DNS), pero sí en las diapositivas: están resumidos en 2.5 por si salen en el test.

Cómo usar esta guía

Resumen de los **Temas 1 y 2** (PEI 1, 30 % de la nota) basado en el resumen previo, revisado y ampliado con *Redes de computadoras: un enfoque descendente* (Kurose & Ross, 7.ª ed.).

- Las cajas **verdes** contienen fórmulas.
- Las cajas **rojas** marcan errores frecuentes de examen.
- Al final hay **ejercicios resueltos**, **problemas de exámenes anteriores** y una **hoja de fórmulas**.
- La última sección lista las **correcciones** hechas respecto al resumen original.

Índice

1 Tema 1: Arquitecturas de red **4**

1.1 Elementos de una red . . . 4

1.1.1 Tecnologías de acceso . . . 4

1.1.2 Medios físicos . . . 5

1.1.3 Clasificación de las redes . . . 5

1.1.4 Red del hogar . . . 5

1.2 Conmutación en el núcleo: circuitos frente a paquetes . . . 6

1.2.1 Multiplexación en circuitos . . . 6
1.2.2 Almacenamiento y reenvío . . . 7
1.2.3 Ventaja de la multiplexación estadística . . . 7
1.3 Estructura de Internet y normalización . . . 7
1.3.1 Estructura de Internet . . . 7
1.3.2 Organismos de normalización . . . 8
1.4 Retardos, pérdidas y tasa de transferencia . . . 8
1.4.1 Retardo nodal . . . 8
1.4.2 Retardo de cola e intensidad de tráfico . . . 9
1.4.3 Retardo extremo a extremo . . . 9
1.4.4 Tasa de transferencia (*throughput*) . . . 9
1.4.5 Producto ancho de banda-retardo . . . 9
1.5 Protocolos y arquitectura en capas . . . 10
1.5.1 Modelo OSI y pila TCP/IP . . . 10
1.5.2 Encapsulamiento . . . 11
1.6 Terminología de capas: servicio, interfaz, PDU, SDU y PCI . . . 11
1.6.1 Funcionalidades genéricas de los protocolos . . . 12
1.7 Servicios conectivos y no conectivos. Circuitos virtuales y datagramas . . . 12
1.8 Problemas de encapsulación, eficiencia y confirmaciones . . . 13

2 Tema 2: Protocolos de la capa de aplicación 15
2.1 Principios de las aplicaciones de red . . . 15
2.1.1 Procesos, sockets y direccionamiento . . . 15
2.1.2 Qué necesita una aplicación del transporte . . . 16
2.2 Servicios de transporte: TCP frente a UDP . . . 16
2.3 La web y HTTP . . . 17
2.3.1 Formato de los mensajes . . . 17
2.3.2 Métodos . . . 18
2.3.3 Códigos de estado . . . 18
2.3.4 Conexiones no persistentes y persistentes . . . 18
2.3.5 Cookies . . . 19
2.3.6 Caché web (servidor proxy) . . . 19
2.3.7 Proxy con GET condicional: ejemplo de examen . . . 19
2.3.8 Tiempos HTTP con tamaños de trama . . . 20
2.3.9 Leer una interacción HTTP . . . 20
2.4 DNS *(Domain Name System)* . . . 21
2.4.1 Jerarquía . . . 21
2.4.2 Consultas iterativas y recursivas . . . 21

2.4.3 Registros de recursos y mensajes DNS . . . 22
2.4.4 Cómo resolver un ejercicio de DNS . . . 22
2.4.5 Preguntas teóricas de DNS que se repiten . . . 24
2.5 Otros protocolos de aplicación (por si salen en test) . . . 25
2.6 Programación con sockets . . . 25
2.6.1 UDP (sin conexión) . . . 25
2.6.2 TCP (orientado a conexión) . . . 26

3 Ejercicios resueltos tipo PEI **28**

4 Problemas de exámenes anteriores **30**
4.1 Retardo, propagación y conmutación de circuitos . . . 30
4.2 Encapsulación, MTU, eficiencia y errores . . . 31
4.3 Tiempos HTTP . . . 32
4.4 Cuestiones teóricas que se repiten . . . 34
4.5 Test: respuestas razonadas . . . 34

5 Hoja de fórmulas **35**

6 Correcciones respecto al resumen original **35**

1. Tema 1: Arquitecturas de red

1.1 Elementos de una red

Internet es una red de redes formada por miles de millones de dispositivos, *enlaces* y *conmutadores de paquetes*. Se describe en tres partes:

- **Borde de la red.** Sistemas finales (*hosts*) que ejecutan aplicaciones. Se agrupan en **clientes** (PC, móviles, dispositivos IoT, que inician peticiones) y **servidores** (máquinas en centros de datos que atienden peticiones y distribuyen contenido).
- **Redes de acceso.** Enlaces que conectan un sistema final con el primer router (*edge router*).
- **Núcleo de la red.** Malla de routers interconectados con enlaces de alta capacidad (fibra óptica terrestre y submarina) que transporta los datos de la red de acceso de origen a la de destino.

Tecnologías de acceso

Tecnología	Medio	Características clave
FTTH *(fibra hasta el hogar)*	Fibra óptica	Fibra hasta la vivienda. Arquitectura típica PON (óptica pasiva): un divisor óptico reparte la fibra entre varios hogares. GPON es asimétrica (en torno a 2,5 Gbps de bajada y 1,25 Gbps de subida); XGS-PON ofrece 10 Gbps simétricos.
HFC *(híbrido fibra-coaxial)*	Fibra hasta el nodo de zona y coaxial hasta el hogar	Medio **compartido** entre los usuarios del vecindario. Usa el estándar **DOCSIS**. Canales de bajada y subida **asimétricos**.
DSL	Par trenzado de cobre de la red telefónica	Línea **dedicada** hasta la central. Con **FDM** se divide en tres bandas: telefonía bidireccional (hasta 4 kHz), subida (4–50 kHz) y bajada (50 kHz–1 MHz). La velocidad cae con la distancia a la central.
Ethernet (IEEE 802.3)	Par trenzado (UTP Cat 5e/6/7) con conectores RJ-45; también fibra	Acceso LAN en empresas, universidades y hogares mediante *switches*. Velocidades de 100 Mbps, 1 Gbps, 10 Gbps o más.
Wi-Fi (IEEE 802.11)	Radio, bandas sin licencia (2,4, 5 y 6 GHz)	Acceso inalámbrico local mediante un punto de acceso. Usa acceso múltiple (CSMA/CA) para compartir el canal.
Celular (4G LTE, 5G NR)	Radio, espectro con licencia	Acceso inalámbrico de área extensa mediante estaciones base (eNodeB / gNodeB), con soporte de movilidad.

Medios físicos

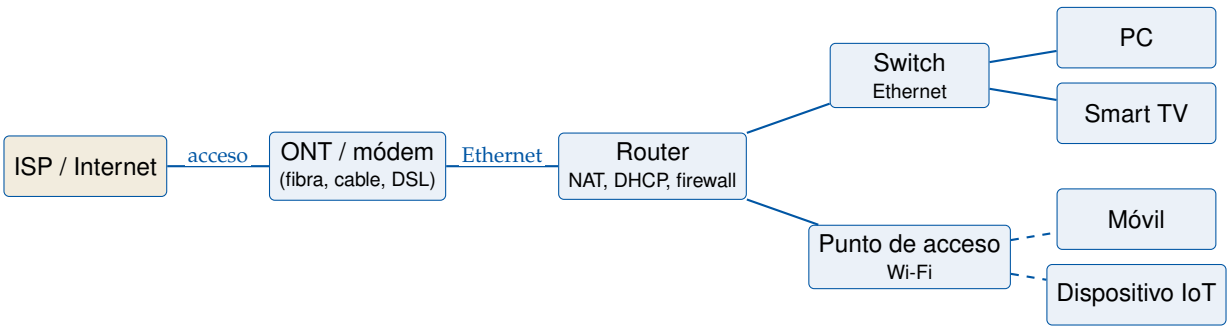
Medio	Observaciones
Par trenzado	Barato y muy extendido. Velocidades de 100 Mbps a 10 Gbps según categoría y longitud.
Cable coaxial	Dos conductores concéntricos. Admite medio compartido y velocidades altas; base de HFC.
Fibra óptica	Pulsos de luz. Muy alta velocidad (decenas o cientos de Gbps), baja atenuación e inmunidad al ruido electromagnético. Es el medio del núcleo.
Radio terrestre	Sin cableado físico; sensible a interferencias, obstáculos y multitrayecto.
Satélite	Geoestacionario (a unos 36 000 km, con retardo de propagación en torno a 280 ms) o de órbita baja, con retardo mucho menor.

Clasificación de las redes

Criterio	Categorías
Topología	**Malla** (cada nodo conectado con varios otros; muy robusta y costosa), **estrella** (todos los nodos a un nodo central; un fallo central deja la red incomunicada), **árbol** (jerarquía de estrellas).
Alcance geográfico	**LAN** (edificio o campus), **MAN** (ciudad o área metropolitana), **WAN** (país o continente).
Cercanía al usuario	**Acceso** (hasta el usuario), **agregación** (concentra el tráfico de muchos accesos), **núcleo** (transporte troncal).
Medio con el usuario	**Cableada** (cobre, fibra) o **radio** (Wi-Fi, celular, satélite).
Movilidad	**Fija** o **móvil**.
Tipo de servicio	Datos, voz, vídeo, servicios interactivos y multimedia.

Red del hogar

Pregunta típica de examen: dibujar el esquema de una red doméstica con acceso a Internet, indicando la función de cada elemento y cómo se interconecta.



Elemento	Función	Interconexión
ONT / módem	Adapta la señal del medio de acceso (fibra, coaxial, par de cobre) a Ethernet.	Hacia el ISP: fibra, coaxial o cobre. Hacia el router: Ethernet.
Router (puerta de enlace)	Enruta entre la red local y el ISP. Hace **NAT** (una IP pública para toda la casa), reparte IP privadas por **DHCP** y filtra con *firewall*.	Un puerto *WAN* y varios *LAN*.
Switch	Conmuta tramas entre los equipos cableados de la LAN (capa 2).	Ethernet (UTP con RJ-45).
Punto de acceso	Da conectividad Wi-Fi a los dispositivos inalámbricos.	Ethernet hacia el router; radio (802.11) hacia los clientes.
Repetidor / PLC	Amplía la cobertura Wi-Fi o lleva la red por la instalación eléctrica.	Radio o cable eléctrico.
Dispositivos	PC, móvil, *smart* TV, consolas, IoT.	Cable Ethernet o Wi-Fi.

En un router doméstico real, router, switch y punto de acceso suelen venir integrados en un solo equipo.

1.2 Conmutación en el núcleo: circuitos frente a paquetes

Parámetro	Conmutación de circuitos	Conmutación de paquetes
Recursos	Reservados extremo a extremo antes de transmitir.	Sin reserva; compartidos bajo demanda.
Garantías	Caudal y retardo garantizados.	Sin garantías (*best effort*); retardo y pérdidas variables.
Multiplexación	**FDM** o **TDM** con recursos dedicados a cada llamada.	**Multiplexación estadística**: los paquetes se intercalan según la demanda.
Ráfagas	Ineficiente: el circuito reservado se desperdicia en los silencios.	Eficiente: aprovecha la inactividad de otros usuarios.
Estado	Establecimiento y cierre de llamada; los conmutadores guardan estado.	Los routers no guardan estado de conexión.
Ejemplo	Red telefónica clásica.	Internet.

Multiplexación en circuitos

- **FDM** *(frequency-division multiplexing)*: el espectro del enlace se divide en bandas fijas; cada conexión usa su banda en exclusiva durante toda la sesión.
- **TDM** *(time-division multiplexing)*: el tiempo se divide en tramas de duración fija, y cada trama en un número fijo de *ranuras*. Cada conexión dispone de una ranura periódica en cada trama.

Almacenamiento y reenvío

Store-and-forward

Un conmutador de paquetes debe recibir el paquete **completo** (los L bits) antes de empezar a transmitir su primer bit por el enlace de salida. Transmitir L bits a velocidad R tarda L/R segundos.

Retardo de un paquete a través de N enlaces (sin colas, procesamiento ni propagación)

$$d_{\text{extremo-extremo}} = N\,\frac{L}{R}$$

(con $N-1$ routers intermedios, todos los enlaces a velocidad R).

Ventaja de la multiplexación estadística

Ejemplo clásico: un enlace de 1 Mbps lo comparten usuarios que, cuando están activos, transmiten a 100 kbps, y que están activos solo el 10 % del tiempo.

- Con **conmutación de circuitos** caben $1\,\text{Mbps}/100\,\text{kbps} = 10$ usuarios, estén activos o no.
- Con **conmutación de paquetes**, con 35 usuarios, la probabilidad de que más de 10 estén activos a la vez es $P(N > 10) \approx 0{,}0004$ (distribución binomial). Casi nunca hay congestión, y se sirve a 3,5 veces más usuarios.

Error frecuente

La conmutación de paquetes *no* es siempre mejor. Para tráfico constante y con exigencias de tiempo real, la reserva de circuitos tiene sentido. Compara siempre tráfico en ráfagas frente a tráfico continuo.

1.3 Estructura de Internet y normalización

Estructura de Internet

- Los usuarios se conectan a **ISP de acceso**. Los ISP se interconectan entre sí, de forma que dos hosts cualesquiera pueden comunicarse.
- Los ISP de nivel superior (*tier-1*) forman la espina dorsal. Los ISP se interconectan directamente (*peering*) o mediante **puntos neutros de intercambio** (IXP).
- Grandes proveedores de contenido operan sus propias redes y centros de datos, acercando el contenido a los usuarios.

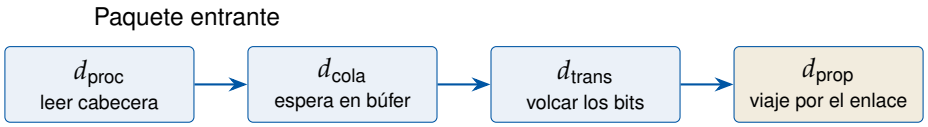
Organismos de normalización

Organismo	Papel
IETF	Desarrolla los estándares de Internet, publicados como documentos **RFC** (*Request For Comments*): TCP, IP, HTTP, SMTP, DNS, etc.
IEEE 802	Estándares de redes locales y metropolitanas: Ethernet (802.3) y Wi-Fi (802.11).
ITU	Estándares de telecomunicaciones (organismo de Naciones Unidas).
ISO	Estándares generales; autora del modelo de referencia OSI.
W3C	Estándares de la web (HTML, CSS, etc.).
ICANN / IANA	Gestión de direcciones IP, nombres de dominio y números de puerto asignados.

La **regulación** (licencias de espectro, competencia entre operadores, obligaciones de servicio) la ejercen organismos públicos nacionales e internacionales; en España, por ejemplo, el ministerio competente en telecomunicaciones y la CNMC. Normalizar sirve para la *interoperabilidad*; regular, para ordenar el mercado y el uso de recursos escasos como el espectro.

1.4 Retardos, pérdidas y tasa de transferencia

Retardo nodal



Retardo nodal

$$d_{nodal} = d_{proc} + d_{cola} + d_{trans} + d_{prop}$$

Componente	Descripción
d_{proc}	Examinar la cabecera, comprobar errores y decidir la interfaz de salida con la tabla de reenvío. Del orden de microsegundos o menos.
d_{cola}	Espera en la cola de salida hasta empezar a transmitirse. **Variable**: depende del tráfico.
$d_{trans} = L/R$	Tiempo de volcar los L bits al enlace a R bps. Depende del **tamaño** y de la **velocidad** del enlace, no de la distancia.
$d_{prop} = d/s$	Tiempo que tarda un bit en recorrer el enlace de longitud d a velocidad de propagación s ($\approx 2 \cdot 10^8$ m/s en fibra y cobre, hasta $3 \cdot 10^8$ m/s en el vacío). Depende de la **distancia**, no del tamaño del paquete.

No confundir d_{trans} con d_{prop}

d_{trans} es el tiempo de *meter* el paquete en el enlace (depende de L y R). d_{prop} es el tiempo de *recorrer* el enlace (depende de d y s). Analogía: una caravana de coches que pasa por un peaje (transmisión) y luego viaja por la carretera (propagación).

Retardo de cola e intensidad de tráfico

Intensidad de tráfico

$$\text{Intensidad de tráfico} = \frac{L \cdot a}{R}$$

L: tamaño medio del paquete (bits); a: tasa media de llegada (paquetes/s); R: velocidad de transmisión (bps).

- Si $La/R \approx 0$: el retardo de cola medio es prácticamente nulo.
- Si $La/R \to 1$: el retardo medio de cola **crece rápidamente** (sin límite en una cola infinita).
- Si $La/R > 1$: llega más tráfico del que el enlace puede sacar. La cola crece sin parar o, si es finita, **se desborda** y se pierden paquetes.

Pérdida de paquetes. Las colas reales son finitas: si un paquete llega a una cola llena, el router lo descarta. El porcentaje de pérdidas crece con la intensidad de tráfico. Un paquete perdido puede retransmitirse extremo a extremo (lo hará TCP, Tema 3).

Retardo extremo a extremo

N enlaces idénticos, sin colas

$$d_{\text{extremo-extremo}} = N\left(d_{\text{proc}} + d_{\text{trans}} + d_{\text{prop}}\right)$$

Se cuentan N transmisiones: la del host origen más las de los $N-1$ routers.

Tasa de transferencia (*throughput*)

- **Instantánea**: velocidad a la que el receptor recibe el archivo en un instante dado.
- **Media**: F/T bps para un archivo de F bits recibido en T segundos.
- En un camino con enlaces de velocidades $R_1, \ldots, R_N$ el cuello de botella limita:

Throughput extremo a extremo

$$\text{throughput} = \min\{R_1, R_2, \ldots, R_N\}, \qquad T = \frac{F}{\text{throughput}}$$

Producto ancho de banda-retardo

Dos definiciones, según el contexto

- En Kurose & Ross (cap. 1) el producto ancho de banda-retardo es $R \cdot d_{\text{prop}}$: el **máximo número de bits que caben en el enlace** a la vez.
- En el contexto de transporte (Tema 3) se usa $R \cdot \text{RTT}$: los bits *en vuelo* que el emisor puede enviar antes de recibir el primer reconocimiento. Determina el tamaño de ventana necesario para llenar la tubería.

Lee el enunciado para ver cuál se pide; en problemas del Tema 1 suele ser $R \cdot d_{\text{prop}}$.

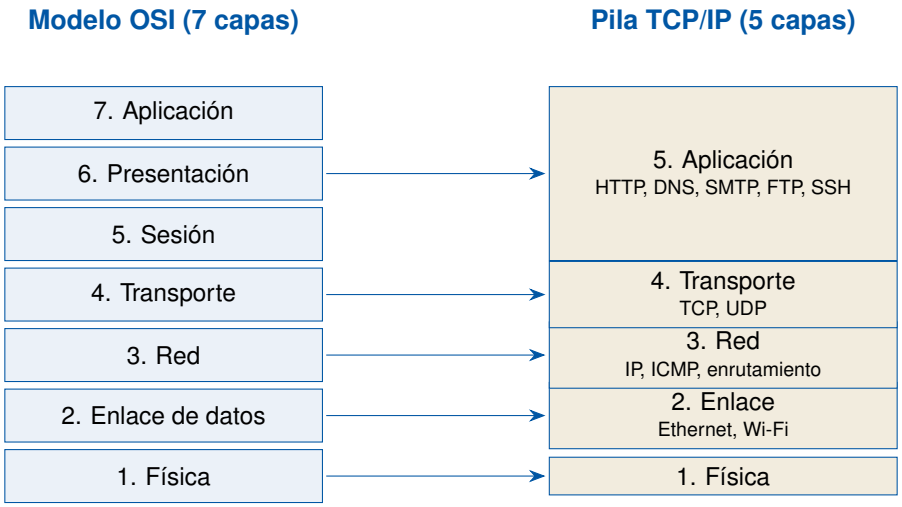
1.5 Protocolos y arquitectura en capas

Protocolo

Un protocolo define el **formato** y el **orden** de los mensajes intercambiados entre entidades pares, y las **acciones** que deben ejecutarse al enviar o recibir un mensaje o al ocurrir un evento (por ejemplo, un temporizador que vence).

La arquitectura en capas divide un problema complejo en partes. Cada capa ofrece un **servicio** a la capa superior apoyándose en los servicios de la inferior, y oculta los detalles de su implementación. Ventajas: modularidad y facilidad de mantenimiento; inconveniente: posible duplicación de funciones y sobrecarga de cabeceras.

Modelo OSI y pila TCP/IP



Capa	PDU	Función
Aplicación	Mensaje	Soporta las aplicaciones de red; define el formato de los mensajes de protocolos como HTTP, DNS o SMTP.
Transporte	Segmento	Transferencia lógica **proceso a proceso**; multiplexa aplicaciones con números de puerto. TCP es fiable y UDP no.
Red	Datagrama IP	Entrega **host a host** a través de varias redes: direccionamiento IP y enrutamiento.
Enlace	Trama (*frame*)	Transfiere tramas entre nodos **adyacentes**; direccionamiento MAC, acceso al medio y detección de errores.
Física	Bit	Transmite bits individuales por el medio (voltajes, frecuencias, tiempos de bit).

Capas de OSI que TCP/IP no distingue

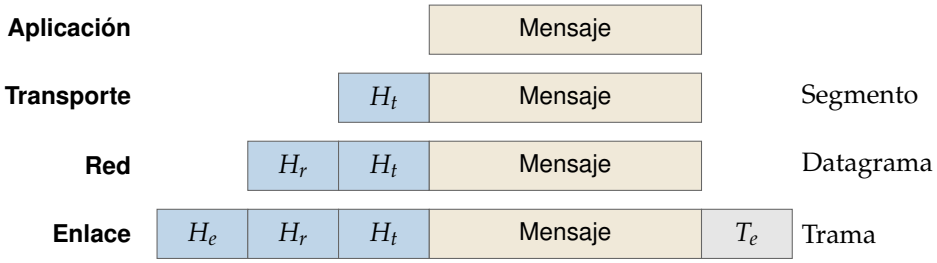
En OSI, *presentación* (cifrado, compresión, formato de datos) y *sesión* (sincronización, recuperación) son capas propias. En TCP/IP, si hacen falta, las implementa la aplicación.

Terminología del PDU de UDP

En Kurose & Ross, la PDU de transporte se llama *segmento* tanto en TCP como en UDP. Muchos textos llaman *datagrama* al paquete UDP. Ambos usos son habituales: usa el término que emplee tu profesor y ten presentes los dos.

Encapsulamiento

Cada capa añade su **cabecera** (y la de enlace, también una cola o *trailer*) al descender por la pila; en el receptor se retiran en orden inverso.



Dispositivos y capas. Un *host* implementa las cinco capas. Un *router* implementa física, enlace y red. Un *switch* de enlace implementa física y enlace.

1.6 Terminología de capas: servicio, interfaz, PDU, SDU y PCI

Término	Significado
Protocolo	Reglas (formato, orden y acciones) que siguen las capas **homólogas** de dos sistemas distintos.
Servicio	Lo que una capa ofrece a la capa inmediatamente superior.
Interfaz	Define qué operaciones y servicios ofrece una capa a la inmediata superior en el **mismo sistema**. Permite sustituir la implementación de una capa sin tocar las demás mientras se mantenga el conjunto de servicios.
PCI *(Protocol Control Information)*	Información de control del protocolo: cabecera (y, en enlace, también cola).
SDU *(Service Data Unit)*	Datos que la capa recibe de la capa superior para transportar.
PDU *(Protocol Data Unit)*	Unidad que intercambian dos entidades pares: PDU = PCI + SDU. La PDU de la capa N es la SDU de la capa $N-1$.
MTU	Tamaño máximo de SDU de la capa de enlace, es decir, de PDU de red que cabe en una trama.

Comunicación horizontal (virtual) y vertical

La **comunicación horizontal o virtual** es la que mantienen dos capas *homólogas* (del mismo nivel) de máquinas diferentes, siguiendo el protocolo de esa capa. Es virtual porque físicamente los datos bajan por la pila, cruzan el medio y suben por la pila del receptor. La **comunicación vertical** es la de capas adyacentes del mismo sistema a través de la interfaz.

Errores frecuentes de test

- Una PDU **no** es idéntica en todas las capas: varía según la capa y el protocolo.
- La PDU de una capa suele ser la SDU de la inmediatamente inferior.
- La especificación de un protocolo puede imponer límites al tamaño de su PDU.
- En OSI **no** existe una capa de "seguridad". Las siete son: física, enlace, red, transporte, sesión, presentación y aplicación.
- TCP/IP no es exclusivo de redes inalámbricas ni de redes locales, y no obliga a usar de forma estricta todas las capas.

Funcionalidades genéricas de los protocolos

Funcionalidad	Para qué sirve
Direccionamiento	Identificar origen y destino (MAC, IP, puerto).
Control de errores	Detectar (suma de comprobación, CRC) y corregir (retransmisión) errores.
Control de flujo	Evitar que el emisor sature al receptor.
Control de congestión	Evitar que el emisor sature la red.
Segmentación y reensamblado	Trocear los datos en PDU del tamaño admitido y recomponerlos.
Multiplexación	Compartir un recurso entre varias comunicaciones (puertos, TDM, FDM).
Establecimiento y cierre de conexión	Preparar y liberar el estado en ambos extremos.
Secuenciación	Numerar las PDU para entregarlas en orden y detectar pérdidas o duplicados.

1.7 Servicios conectivos y no conectivos. Circuitos virtuales y datagramas

Desde el punto de vista de la **capa de red** (OSI), el servicio puede ser:

	Conectivo (circuitos virtuales)	No conectivo (datagramas)
Establecimiento	Hay fase previa de establecimiento de la conexión lógica. Retrasa el inicio de la transferencia.	No hay conexión lógica: se transmite de inmediato (mayor *prontitud*).
Estado en los nodos	Los nodos guardan **estado** de cada conexión (tabla de circuitos virtuales).	Los nodos **no** guardan estado.
Encaminamiento	La ruta se fija al establecer la conexión y todos los paquetes la siguen. Cada paquete lleva un identificador de circuito virtual.	Cada paquete se encamina de forma independiente según la **dirección de destino**.
Orden	Garantiza la **secuencia** de las PDU.	Puede entregar desordenado.
Procesamiento por nodo	Menor, porque el camino ya está decidido.	Mayor demora de procesamiento por nodo, porque cada PDU es independiente.
Robustez	Un fallo de un nodo de la ruta tumba la conexión.	Encuentra rutas alternativas con facilidad.
Ejemplos	X.25, Frame Relay, ATM, MPLS.	**IP** (Internet).

Circuito virtual no es conmutación de circuitos

Un circuito virtual sigue siendo conmutación de **paquetes**: los recursos no se reservan en exclusiva, puede haber **congestión** y los retardos **no son constantes**. Solo comparte con la conmutación de circuitos que la ruta es fija y que hay establecimiento previo. Un servicio no conectivo en capa de red **no** implementa control de errores ni de flujo (IP no lo hace): eso, si hace falta, lo aporta el transporte.

1.8 Problemas de encapsulación, eficiencia y confirmaciones

Son el tipo de ejercicio más frecuente en la PEI 1. Método:

1. **SDU máxima en cada capa**, de abajo arriba. Si MTU es el máximo de SDU del enlace, la PDU de red cabe entera: $\text{SDU}_{\text{red}} = \text{MTU} - \text{PCI}_{\text{red}}$ y $\text{SDU}_{\text{transp}} = \text{SDU}_{\text{red}} - \text{PCI}_{\text{transp}}$.
2. **Número de PDU**: $\lceil \text{datos de aplicación} / \text{SDU}_{\text{transp}} \rceil$. La última puede ser más corta.
3. **Tamaño de trama**: datos + PCI de transporte + PCI de red + PCI de enlace (cabecera y cola).
4. **Retardos**: $d_{\text{trans}} = L_{\text{trama}}/R$ y $d_{\text{prop}} = d/v$.
5. Dibujar el **diagrama de tiempos** y sumar sobre el diagrama.

Definiciones de la asignatura: eficiencia, caudal y errores

TBCF: total de bits de capa física necesarios para transmitir todos los bits de la SDU_N (trama completa, con cabecera y cola de enlace).

$$E_N = \frac{\text{SDU}_N}{\text{TBCF}}, \qquad C_N = \frac{\text{SDU}_N}{T_N}, \qquad P_{ET} = 1 - (1 - \text{BER})^L$$

E_{red} usa la SDU_{red} (el segmento entero, con su cabecera de transporte); E_{transp} y E_{aplic} usan los

datos de aplicación. T_N es el tiempo total de transmitir y recibir la SDU_N.

$$\text{tramas totales con errores} \approx \text{tramas} \cdot (1 + P_{ET}), \qquad E_{\text{con errores}} \approx \frac{\text{SDU}_N}{\text{TBCF}\,(1 + P_{ET})}$$

$$\text{tramas/s} = \frac{R}{L}, \qquad \text{tramas erróneas/s} = \frac{R}{L}\,P_{ET}$$

Longitud de trama para que el primer bit llegue cuando sale el último: $L = R \cdot d_{\text{prop}}$. La aproximación $(1 + P_{ET})$ cuenta una retransmisión por cada trama errónea. El valor exacto, con reintentos hasta acertar, sería $E \cdot (1 - P_{ET})$; difieren poco.

Origen de estas definiciones

Las he tomado de las respuestas de tu cuaderno de NotebookLM, que las cita del documento `GP12_2_Definiciones.pdf` (pág. 2). No he visto ese archivo, así que compruébalo ahí.

Tiempo total con n fragmentos de L bits (ACK de tamaño despreciable)

Envío continuo, un ACK final	$T = nL/R + 2\,d_{\text{prop}}$
Parada y espera (ACK de cada fragmento)	$T = n\left(L/R + 2\,d_{\text{prop}}\right)$
ACK cada k fragmentos, el emisor espera	$T = \lceil n/k \rceil \left(kL/R + 2\,d_{\text{prop}}\right)$ (si el último bloque es más corto, ajusta su k)
ACK cada k fragmentos, sin esperar (dúplex)	$T = nL/R + 2\,d_{\text{prop}}$ (el último ACK sale al recibir el último fragmento)

Si el ACK tiene tamaño no despreciable, suma además L_{ACK}/R por cada ACK que el emisor tenga que esperar. Caudal = bits de aplicación entregados/T.

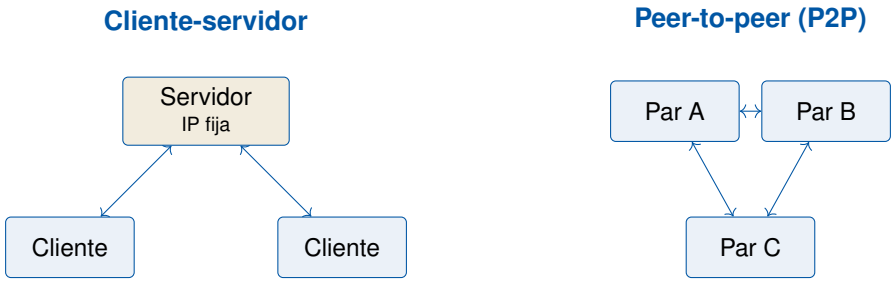
Sobre el ACK y la propagación

Cuando el ACK es despreciable, cada ida y vuelta añade $2\,d_{\text{prop}}$. Fíjate siempre en si el emisor se queda *bloqueado* esperando el ACK (suma el tiempo) o si sigue enviando (el ACK solo añade la propagación final).

2. Tema 2: Protocolos de la capa de aplicación

2.1 Principios de las aplicaciones de red

Una aplicación de red está formada por programas que se ejecutan en distintos sistemas finales y se comunican a través de la red. No se programa código para los routers del núcleo.



	Cliente-servidor	P2P
Servidor	Siempre activo, con **IP fija** y conocida; suele ser una granja de servidores.	Sin servidor dedicado permanente.
Clientes / pares	Se conectan de forma intermitente, con IP posiblemente dinámica; **no se comunican entre sí**.	Se conectan de forma intermitente, cambian de IP y **se comunican directamente**.
Escalabilidad	Limitada por el servidor y su ancho de banda.	**Autoescalable**: cada par que entra aporta capacidad (descarga y subida).
Dificultad	Sencilla de gestionar.	Gestión compleja: seguridad, incentivos, conectividad.

Tiempo de distribución de un archivo de F bits a N pares

u_s: subida del servidor; u_i: subida del par i; $d_{\min}$: menor bajada entre los pares.

$$D_{\text{CS}} \geq \max\left\{\frac{NF}{u_s}, \frac{F}{d_{\min}}\right\}, \qquad D_{\text{P2P}} \geq \max\left\{\frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum_{i=1}^{N} u_i}\right\}$$

En cliente-servidor el tiempo crece *linealmente* con N; en P2P la capacidad de subida total crece con N, y el tiempo tiende a un valor acotado.

Procesos, sockets y direccionamiento

- Se comunican **procesos**, no máquinas. El que inicia la comunicación es el **cliente**; el que espera, el **servidor**.
- El **socket** es la interfaz (API) entre el proceso de aplicación y la capa de transporte del sistema operativo.
- Para identificar un proceso destino se necesitan dos datos: la **dirección IP** del host (32 bits en IPv4, 128 en IPv6) y el **número de puerto** (16 bits, 0–65 535).
- *Puertos bien conocidos* (0–1023), asignados por IANA: HTTP 80, HTTPS 443, DNS 53, SSH 22, SMTP 25.

Qué necesita una aplicación del transporte

Requisito	Ejemplo
Transferencia fiable de datos	Imprescindible en correo, transferencia de archivos y web; tolerable cierta pérdida en audio y vídeo en tiempo real.
Tasa de transferencia	Aplicaciones *sensibles al ancho de banda* (vídeo) frente a *elásticas* (correo, web).
Temporización	Telefonía y juegos en red exigen retardos bajos.
Seguridad	Confidencialidad, integridad, autenticación (cifrado con TLS).

2.2 Servicios de transporte: TCP frente a UDP

	TCP	UDP
Conexión	Orientado a conexión: *handshake* de tres vías antes de enviar datos.	Sin conexión: se envía directamente.
Fiabilidad	Sin pérdidas, sin duplicados, en orden.	No fiable: sin garantías de entrega ni de orden.
Control de flujo	Sí (ventana de recepción).	No.
Control de congestión	Sí.	No.
Cabecera	20 bytes como mínimo.	**8 bytes**.
Garantías de tasa o retardo	**Ninguna**.	**Ninguna**.
Usos típicos	Web, correo, archivos, SSH.	DNS, VoIP, streaming, juegos en red.

Cabecera UDP (cuatro campos de 16 bits):

Puerto origen	**Puerto destino**
Longitud	**Suma de comprobación**
Datos de aplicación	

TCP en una frase. Garantiza que los bytes llegan sin errores, sin huecos, sin duplicados y en orden, mediante números de secuencia, reconocimientos acumulativos (ACK) y temporizador de retransmisión (RTO); el control de flujo evita saturar el receptor (ventana de recepción `rwnd`) y el control de congestión protege la red (ventana de congestión `cwnd`). Se estudia en detalle en el Tema 3.

Error frecuente

TCP *no* garantiza ancho de banda mínimo ni retardo máximo. Ofrece fiabilidad, no rendimiento garantizado. Y los servicios de seguridad (TLS) no forman parte de TCP: se añaden en la capa de aplicación.

Aplicación	Protocolo de aplicación	Transporte
Web	HTTP, HTTPS	TCP
Correo electrónico	SMTP	TCP
Resolución de nombres	DNS	UDP (TCP para transferencias de zona)
Transferencia de archivos	FTP	TCP
Acceso remoto	SSH	TCP
Telefonía por Internet	Propietario (p. ej., RTP)	Normalmente UDP

2.3 La web y HTTP

HTTP *(HyperText Transfer Protocol)*

Protocolo de aplicación de la web, con el modelo cliente-servidor: el navegador pide objetos (HTML, imágenes, scripts…) y el servidor web los devuelve. Cada objeto se identifica con una URL. Usa **TCP**, puerto 80 (443 en HTTPS). Es un protocolo **sin estado**: el servidor no guarda información de peticiones anteriores del cliente.

Formato de los mensajes

Los mensajes son texto ASCII legible. **Petición**:

```
GET /index.html HTTP/1.1          <- linea de peticion
Host: www.uah.es                  <- cabeceras
Connection: close
User-Agent: Mozilla/5.0
Accept-Language: es
                                  <- linea en blanco (fin de cabeceras)
```

Respuesta:

```
HTTP/1.1 200 OK                   <- linea de estado
Connection: close
Date: Sat, 07 Nov 2026 10:00:00 GMT
Server: Apache
Last-Modified: Fri, 06 Nov 2026 09:00:00 GMT
Content-Length: 6821
Content-Type: text/html
                                  <- linea en blanco
(datos: el objeto solicitado)     <- cuerpo
```

Métodos

Método	Uso
`GET`	Pide el objeto identificado por la URL. Los parámetros pueden ir en la propia URL.
`POST`	Envía datos (por ejemplo, un formulario) en el cuerpo del mensaje.
`HEAD`	Como GET, pero el servidor omite el cuerpo; útil para depuración.
`PUT`	Sube un objeto al servidor, reemplazando el recurso de la URL.
`DELETE`	Borra el objeto de la URL en el servidor.

Códigos de estado

Código	Nombre	Significado
200	OK	Petición correcta; el objeto va en la respuesta.
301	Moved Permanently	Recurso movido de forma definitiva; la nueva URL va en la cabecera `Location`.
304	Not Modified	La copia en caché del cliente está al día; no se envía el cuerpo.
400	Bad Request	Petición malformada.
404	Not Found	El recurso no existe en el servidor.
418	I'm a teapot	Código humorístico del RFC 2324 (HTCPCP, 1 de abril de 1998): "soy una tetera" y me niego a preparar café. Sigue reservado en el estándar HTTP actual (RFC 9110) y algunos servidores lo usan como guiño o para rechazar bots. Es un error del cliente (4xx).
500	Internal Server Error	Fallo interno genérico del servidor.
503	Service Unavailable	Servidor temporalmente no disponible (sobrecarga o mantenimiento).
505	HTTP Version Not Supported	Versión de HTTP no soportada.

Regla de memoria: **2xx** éxito, **3xx** redirección, **4xx** error del cliente, **5xx** error del servidor.

Conexiones no persistentes y persistentes

- **No persistente** (HTTP/1.0): se abre y cierra una conexión TCP **por cada objeto**. Cada objeto cuesta 2 RTT más su tiempo de transmisión: un RTT para el *handshake* TCP y otro para la petición y los primeros bytes de la respuesta. Obliga a reservar *buffers* y variables TCP por cada conexión, con mucha carga para el servidor.
- **Persistente** (HTTP/1.1, modo por defecto): el servidor deja abierta la conexión TCP tras responder, y las peticiones siguientes del mismo cliente la reutilizan. Con **procesamiento en cadena** (*pipelining*) el cliente envía varias peticiones sin esperar las respuestas anteriores.

Tiempo de respuesta de un objeto con conexión no persistente

$$T = 2\,\mathrm{RTT} + t_{\text{transmisión del objeto}}$$

Ejemplo: página con **1 archivo base y 10 imágenes** (tiempos de transmisión despreciables, sin conexiones paralelas):

Modo	Tiempo
No persistente	$11 \cdot 2\,\text{RTT} = 22\,\text{RTT}$
Persistente sin *pipelining*	$1 + 1 + 10 = 12\,\text{RTT}$
Persistente con *pipelining*	$1 + 1 + 1 = 3\,\text{RTT}$

(En el modo persistente: 1 RTT de *handshake*, 1 RTT para el archivo base y después 1 RTT por imagen sin *pipelining* o 1 RTT en total para todas con *pipelining*.)

HTTP/2

HTTP/2 mantiene la semántica de HTTP/1.1 pero permite *entrelazar* varias peticiones y respuestas en una misma conexión, mitigando el bloqueo de cabeza de línea. El temario del curso se centra en HTTP/1.x.

Cookies

HTTP no tiene estado, pero los sitios necesitan identificar a los usuarios (carritos, banca). Las cookies tienen **cuatro componentes**:

1. Una cabecera `Set-Cookie:` en la **respuesta** del servidor.
2. Una cabecera `Cookie:` en las **peticiones** posteriores del cliente.
3. Un **archivo de cookies** guardado en el equipo del usuario y gestionado por el navegador.
4. Una **base de datos en el servidor** que asocia el identificador con el perfil del usuario.

Caché web (servidor proxy)

Un **proxy caché** atiende peticiones en nombre del servidor de origen y guarda copias de los objetos recientes. Reduce el tiempo de respuesta, el tráfico en el enlace de acceso de la organización y la carga de los servidores de origen.

Retardo medio con caché

$$d_{\text{medio}} = h \cdot d_{\text{caché}} + (1 - h) \cdot d_{\text{origen}}$$

h: tasa de aciertos (*hit rate*). Ejemplo: $h = 0{,}4$, $d_{\text{caché}} = 10$ ms y $d_{\text{origen}} = 2$ s dan $0{,}4 \cdot 0{,}01 + 0{,}6 \cdot 2 = 1{,}204$ s.

GET condicional. Para no servir objetos obsoletos, la caché pide al origen el objeto con la cabecera `If-Modified-Since: <fecha>`. Si no ha cambiado, el servidor responde `304 Not Modified` sin cuerpo.

Proxy con GET condicional: ejemplo de examen

Proxy HTTP/1.1 sin *pipelining*, con la solicitud condicionada activada y caché vacía. La página `www.ojo.org` tiene un HTML y dos PNG, todos con `Last-Modified: 21/10/2022 12:20:30`. Un primer usuario la pide. Más tarde, un segundo usuario la pide de nuevo y solo el segundo PNG ha cambiado (`3/11/2022 15:20:30`).

Orden	Mensaje	Contenido
Primer usuario (la caché está vacía, todo va al origen)		
1–4	U1 → proxy → origen → proxy → U1	`GET html`; `200 OK` con `Last-Modified: 21/10/2022 12:20:30`. El proxy lo guarda.
5–8	Igual para `png1`	`GET png1`; `200 OK`. Se guarda.
9–12	Igual para `png2`	`GET png2`; `200 OK`. Se guarda.
Segundo usuario (el proxy valida cada objeto con el origen)		
1–4	U2 → proxy; proxy → origen; origen → proxy; proxy → U2	U2: `GET html`. Proxy: `GET html` con `If-Modified-Since: 21/10/2022 12:20:30`. Origen: `304 Not Modified`. Proxy a U2: `200 OK` con el HTML de su caché.
5–8	Igual para `png1`	Origen: `304 Not Modified`. Proxy a U2: `200 OK` con la copia de caché.
9–12	Igual para `png2`	Origen: `200 OK` con el objeto nuevo y `Last-Modified: 3/11/2022 15:20:30`. El proxy actualiza su caché y reenvía `200 OK` a U2.

Un `304` no lleva cuerpo, así que ahorra ancho de banda en el enlace hacia el origen. El cliente final recibe siempre `200 OK` del proxy, porque su petición no era condicional.

Tiempos HTTP con tamaños de trama

Cuando el enunciado da tamaños de mensajes HTTP, tasa binaria y retardo de propagación, y dice que se ignore el establecimiento TCP:

1. Calcula la **trama** de cada mensaje: tamaño HTTP + cabeceras de transporte, red y enlace (con la cola si la hay). Comprueba que cabe en la MTU; si no, fragmenta.
2. $d_{\text{trans}} = L_{\text{trama}} / R$ para cada mensaje.
3. **Establecimiento TCP**: si el enunciado dice que se ignore, no cuenta. Si no lo dice, cuenta 1 RTT por conexión, y el primer GET viaja con el tercer mensaje del *handshake*. (Según la solución de PP3 que cita tu cuaderno; compruébalo.)
4. **Sin *pipelining***: cada objeto cuesta $d_{\text{trans}}(\text{GET}) + d_{\text{prop}} + d_{\text{trans}}(\text{resp.}) + d_{\text{prop}}$, uno detrás de otro. Primero va el HTML, que revela qué objetos hay que pedir.
5. **Con *pipelining***: tras recibir el HTML se envían todos los GET seguidos. El servidor responde según le van llegando y no puede transmitir dos respuestas a la vez por el mismo enlace.

Leer una interacción HTTP

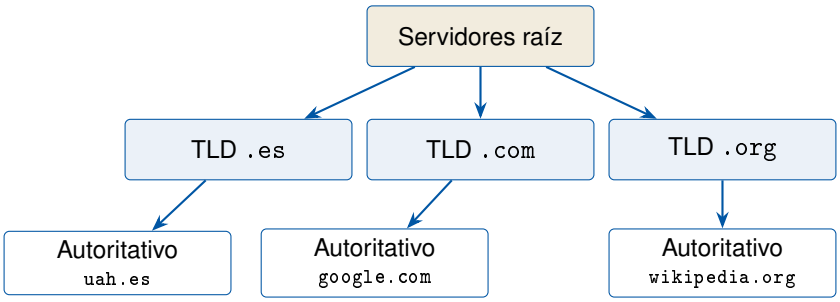
- **Método**: el de la primera línea de la solicitud (`GET`).
- **Código de estado**: el de la primera línea de la respuesta (`200 OK`: petición correcta, el objeto va en el cuerpo).
- `If-Modified-Since` en un GET condicional: si la fecha es **anterior** a `Last-Modified`, el objeto ha cambiado y el servidor responde `200 OK` con el objeto, que es el caso del ejemplo del examen (la respuesta lleva cuerpo). Si fuese **igual o posterior**, respondería `304 Not Modified` sin cuerpo. Si se omite la cabecera, la petición no es condicional y el servidor devuelve el objeto.

2.4 DNS *(Domain Name System)*

DNS es una **base de datos distribuida y jerárquica** y, a la vez, un protocolo de aplicación que traduce nombres (`www.uah.es`) a direcciones IP. Funciona sobre **UDP, puerto 53** (TCP para transferencias de zona).

Servicios adicionales: alias de host (CNAME), alias de servidor de correo (MX) y **distribución de carga** entre servidores web replicados (varias IP para un mismo nombre).

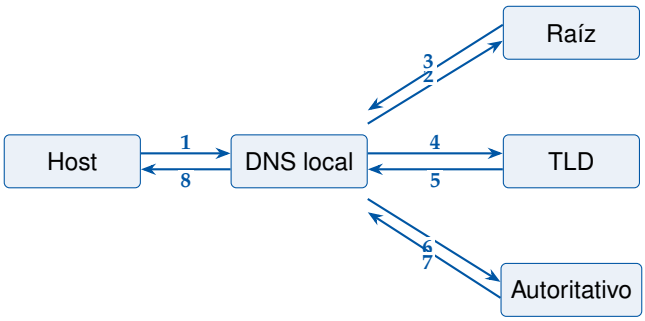
Jerarquía



- **Raíz:** 13 conjuntos lógicos de servidores (replicados en cientos de instancias). Devuelven los servidores TLD responsables de cada extensión.
- **TLD** *(top-level domain)*: dominios genéricos (`.com`, `.org`, `.net`...) y de país (`.es`, `.uk`...). Devuelven el servidor autoritativo de la organización.
- **Autoritativos:** de organizaciones o ISP; contienen los registros definitivos de nombres a IP.
- **DNS local** (*resolver*): del ISP o de la red corporativa. No pertenece a la jerarquía, pero es el primer interlocutor del host; responde desde su **caché** o inicia la búsqueda.

Consultas iterativas y recursivas

Iterativa (el DNS local hace todo el recorrido)



Recursiva (cada servidor delega y espera la respuesta)



- **Iterativa:** el servidor consultado, si no sabe la respuesta, devuelve la **dirección del siguiente servidor** a quien pregunta. Es quien pregunta (el DNS local) quien recorre la jerarquía.

- **Recursiva:** el servidor consultado se encarga de obtener la respuesta final: reenvía la petición y no contesta hasta tenerla. Carga más a los servidores altos de la jerarquía.
- En la práctica, la consulta del host a su DNS local suele ser **recursiva**, y las del DNS local hacia raíz, TLD y autoritativo suelen ser **iterativas**. Ambas variantes son válidas en teoría.

Caché y TTL. Los servidores guardan en caché las respuestas durante el tiempo indicado por el **TTL** del registro. Así, la mayoría de las consultas se resuelven sin llegar a la raíz.

Registros de recursos y mensajes DNS

Un registro de recurso (RR) es una tupla **(Name, Value, Type, TTL)**:

Type	Name	Value
A	Nombre de host.	Dirección **IPv4** (32 bits), por ejemplo `193.146.56.2`.
AAAA	Nombre de host.	Dirección **IPv6** (128 bits).
NS	Dominio (`uah.es`).	Nombre del servidor de nombres **autoritativo** de ese dominio.
CNAME	Alias (`www.ejemplo.com`).	Nombre **canónico** real (`srv1.este.ejemplo.com`).
MX	Dominio.	Nombre canónico del **servidor de correo** del dominio.

Mensajes DNS. Consultas y respuestas usan el mismo formato: cabecera de **12 bytes** (identificador de 16 bits que casa consulta y respuesta, indicadores como *consulta/respuesta*, *autoritativa*, *recursión deseada* y *recursión disponible*, y contadores de cada sección), seguida de las secciones de preguntas, respuestas, autoridad e información adicional.

Cómo resolver un ejercicio de DNS

Qué sabe cada servidor y qué indican los *flags*

- **Raíz**: devuelve la IP del servidor TLD de la extensión (`.es`, `.com`...).
- **TLD**: devuelve el NS y la IP (registro A) del servidor autoritativo del dominio.
- **Autoritativo**: devuelve la IP definitiva (A), el alias (CNAME) o el servidor de correo (MX). Si pides un nombre con CNAME, la respuesta trae el CNAME y el A del nombre canónico.
- **RD** *(recursion desired)*: lo activa el que pregunta si quiere resolución recursiva. **RA** *(recursion available)*: lo activa el servidor que *ofrece* recursión. Si ningún servidor la ofrece, RA vale 0 en todas las respuestas.
- **AA** *(authoritative answer)*: vale 1 solo si responde el servidor autoritativo del dominio. Una respuesta de caché o de un TLD o servidor local no es autoritativa.
- **Caché**: tras una resolución, cada servidor que intervino guarda lo que supo (durante el TTL). En la siguiente consulta se saltan los pasos ya conocidos.

Tres servidores A, B, C. El cliente conoce A, A conoce B, B conoce C y C tiene el registro de `www.examen.com`:

1.ª recursiva, resto iterativas	Todas iterativas	Todas recursivas
1. Cliente → A: consulta	1. Cliente → A: consulta	1. Cliente → A: consulta
2. A → B: consulta	2. A → Cliente: "pregunta a B"	2. A → B: consulta
3. B → A: "pregunta a C"	3. Cliente → B: consulta	3. B → C: consulta
4. A → C: consulta	4. B → Cliente: "pregunta a C"	4. C → B: IP
5. C → A: IP	5. Cliente → C: consulta	5. B → A: IP
6. A → Cliente: IP	6. C → Cliente: IP	6. A → Cliente: IP

Ejemplo con registros (`vacaciones.com`). El servidor autoritativo `dns.vacaciones.com` (`58.59.60.63`) tiene `www CNAME vacaciones.com`, `correo A 58.59.60.62`, `vacaciones.com MX correo.vacaciones.com` y `vacaciones.com A 58.59.60.61`. El TLD `.com` (`1.2.3.4`) tiene `vacaciones.com NS dns.vacaciones.com` y `dns.vacaciones.com A 58.59.60.63`. Un usuario con DNS local `8.8.8.8` pide `www.vacaciones.com` con cachés vacías.

a) Local recursivo, resto iterativos	b) Todos recursivos (incluido el local)
1. PC → local: `www.vacaciones.com`	1. PC → local: `www.vacaciones.com`
2. local → raíz: ídem	2. local → raíz: ídem
3. raíz → local: TLD `.com` = `1.2.3.4`	3. raíz → TLD: ídem
4. local → TLD: ídem	4. TLD → `dns.vacaciones.com`: ídem
5. TLD → local: NS `dns.vacaciones.com`, A `58.59.60.63`	5. `dns` → TLD: CNAME `vacaciones.com`, A `58.59.60.61` (AA = 1)
6. local → `58.59.60.63`: ídem	6. TLD → raíz: ídem
7. `dns` → local: CNAME `vacaciones.com`, A `58.59.60.61` (AA = 1)	7. raíz → local: ídem
8. local → PC: ídem (RA = 1)	8. local → PC: ídem

c) Enviar un correo a `reservas@vacaciones.com` tras el caso a): el cliente necesita el registro **MX** de `vacaciones.com`. El local ya conoce el autoritativo por la caché, así que son 4 mensajes: PC → local (MX de `vacaciones.com`); local → `58.59.60.63`; `dns` → local (MX `correo.vacaciones.com` y A `58.59.60.62`); local → PC.

Con nombres de otra web (`mispelis.es`**)**, local recursivo, raíz iterativa, TLD recursivo y autoritativo iterativo:

1. PC → local: `www.mispelis.es`. 2. local → raíz: ídem. 3. raíz → local: IP `es`.
4. local → TLD `.es`: ídem. 5. TLD → autoritativo (IP `mispelis`): ídem. 6. autoritativo → TLD: IP `web`.
7. TLD → local: IP `web`. 8. local → PC: IP `web`.

Después, para `servidor-streaming.mispelis.es`: el local ya tiene la IP del TLD en caché y se salta la raíz. Son 6 mensajes: PC → local; local → TLD; TLD → autoritativo; autoritativo → TLD; TLD → local; local → PC. El local no conoce al autoritativo, porque el TLD recursivo solo le devolvió la IP final.

Todos iterativos, incluido el DNS local (RA = 0 en todas las respuestas): el PC hace todo el recorrido. Local devuelve la IP de la raíz, el PC pregunta a la raíz (IP del TLD), al TLD (IP del

autoritativo) y al autoritativo (IP del servidor web, AA = 1). Para un segundo dominio, el PC reutiliza la raíz y el TLD de su caché.

Qué descarga un navegador: las etiquetas `<img>`, `<video>` o `<script>` se descargan automáticamente. Un enlace `<a href>` solo se pide si el usuario hace clic.

Preguntas teóricas de DNS que se repiten

Pregunta	Respuesta corta
Puerto y transporte de DNS	Puerto 53; **UDP** para consultas y respuestas normales y TCP para transferencias de zona.
4 clases de servidores y jerarquía	**Raíz** → **TLD** → **autoritativos** (de organizaciones o ISP). A ellos se suma el **DNS local** (fuera de la jerarquía), primer interlocutor del host.
Servidor TLD	Mantiene los dominios de primer nivel (`.com`, `.es`...) y sabe qué servidor autoritativo responde de cada dominio de segundo nivel.
Registro A y AAAA	A asocia un nombre con una IPv4 (32 bits); AAAA, con una IPv6 (128 bits).
CNAME	Alias: apunta un nombre a su nombre canónico, por ejemplo `www.ejemplo.com` a `srv1.ejemplo.com`. Permite cambiar el servidor real sin tocar el alias.
Otros registros	NS (servidor autoritativo del dominio), MX (servidor de correo), PTR (resolución inversa).
Búsqueda directa e inversa	Directa: nombre → IP. Inversa: IP → nombre (registro PTR).
Dos tipos de mensaje	**Consulta** y **respuesta**. Tienen el **mismo formato**: cabecera de 12 bytes con identificador, *flags* y contadores, y las secciones de pregunta, respuesta, autoridad e información adicional. Se diferencian en el bit de consulta/respuesta y en que la respuesta rellena las secciones de respuesta, autoridad e información adicional con RR. Comparten el identificador y repiten la pregunta.

2.5 Otros protocolos de aplicación (por si salen en test)

Protocolo	Puerto y transporte	Claves
FTP	TCP, 21 (control) y 20 (datos)	Dos conexiones TCP: la de **control** (comandos, fuera de banda, persistente) y una de **datos** por cada archivo. Modo *activo* (el servidor abre la conexión de datos al cliente) o *pasivo* (la abre el cliente). Con estado: el servidor guarda la sesión.
SMTP	TCP, 25	Envía correo entre servidores y desde el agente de usuario (*push*). Comandos `HELO`, `MAIL FROM`, `RCPT TO`, `DATA` y `QUIT`; el cuerpo termina con una línea con solo un punto. Conexión persistente.
POP3	TCP, 110	Recupera el correo del buzón (*pull*). Tres fases: **autorización** (`user`, `pass`), **transacción** (`list`, `retr`, `dele`) y **actualización** (`quit`).
IMAP	TCP, 143	Recupera correo y mantiene carpetas y estado en el servidor.

El correo web usa HTTP entre el navegador y el servidor del buzón. Entre servidores sigue usándose SMTP. Para enviar a `usuario@dominio` el servidor pregunta por DNS el registro **MX** y después el A del servidor de correo.

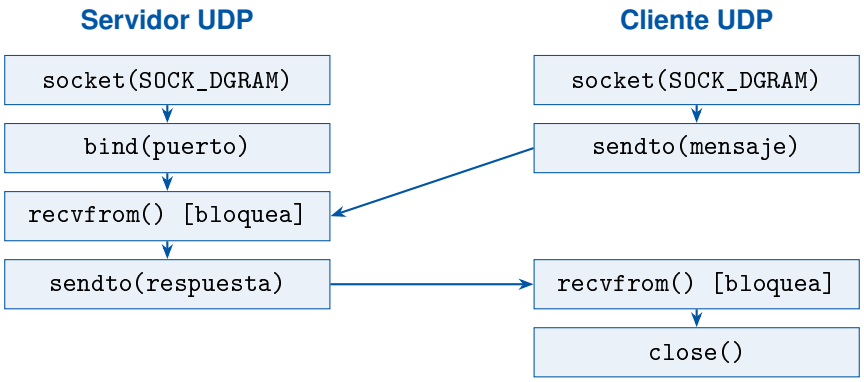
BitTorrent (P2P). Los pares que reparten un archivo forman un **torrent**, y se pasan *fragmentos* del mismo tamaño (típicamente 256 KB). Un **tracker** informa a cada par que entra de otros pares. Cada par pide primero los fragmentos más raros entre sus vecinos (*rarest first*) y reparte la subida con una estrategia de *tit-for-tat*: envía a los cuatro vecinos que más le aportan, y de vez en cuando a uno al azar.

2.6 Programación con sockets

El **socket** es la interfaz entre la aplicación y la capa de transporte. Con ejemplos en Python 3 (modelo de Kurose & Ross: el cliente envía una frase en minúsculas y el servidor la devuelve en mayúsculas).

UDP (sin conexión)

No hay *handshake*: cada datagrama lleva explícitamente la IP y el puerto de destino. El servidor obtiene la dirección del cliente de cada datagrama recibido.



Listing 1: UDPServer.py

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print('El servidor esta listo para recibir')
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(), clientAddress)
```

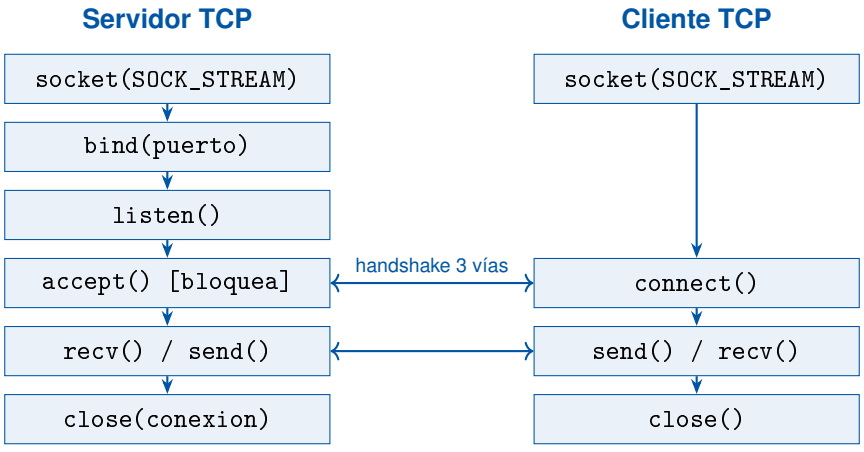
Listing 2: UDPClient.py

```
from socket import *
serverName = 'localhost'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM)
message = input('Escribe una frase en minusculas: ')
clientSocket.sendto(message.encode(), (serverName, serverPort))
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

El cliente no llama a `bind`: el sistema operativo le asigna un **puerto efímero**.

TCP (orientado a conexión)

Antes de intercambiar datos hay que establecer la conexión. El servidor tiene dos tipos de socket: el **socket de bienvenida**, que escucha, y un **socket de conexión** nuevo, creado por `accept()` para cada cliente.



Listing 3: TCPServer.py

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
print('El servidor esta listo para recibir')
while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
```

```
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()
```

Listing 4: TCPClient.py

```
from socket import *
serverName = 'localhost'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Escribe una frase en minusculas: ')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print('Del servidor:', modifiedSentence.decode())
clientSocket.close()
```

	UDP	TCP
Tipo de socket	`SOCK_DGRAM`	`SOCK_STREAM`
Servidor	`bind`, `recvfrom`, `sendto`	`bind`, `listen`, `accept`, `recv`, `send`
Cliente	`sendto`, `recvfrom` (sin `connect`)	`connect`, `send`, `recv`
Dirección destino	En **cada** envío.	Se fija una vez, en `connect`.
Sockets en el servidor	**Uno** para todos los clientes.	Uno de bienvenida y **uno por cliente**.
Datos	Mensajes (datagramas) independientes.	**Flujo de bytes** ordenado y fiable.

Error frecuente

En TCP, `accept()` **no** cierra el socket de bienvenida: sigue escuchando mientras se atiende al cliente por el socket de conexión. Y TCP es un flujo de bytes: un `send` de n bytes puede llegar como varios `recv`.

3. Ejercicios resueltos tipo PEI

Ejercicio 1: retardo extremo a extremo

Un paquete de 8000 bits recorre dos enlaces idénticos (host → router → host), cada uno de 2 Mbps y 1000 km, con velocidad de propagación $2 \cdot 10^8$ m/s. Se desprecian el procesamiento y las colas. Calcula el retardo extremo a extremo.

Solución. $d_{\text{trans}} = \dfrac{8000}{2 \cdot 10^6} = 4$ ms, $\quad d_{\text{prop}} = \dfrac{10^6}{2 \cdot 10^8} = 5$ ms.
Con $N = 2$ enlaces: $d = 2\,(4 + 5) = \mathbf{18\ ms}$.

Ejercicio 2: store-and-forward

Un paquete de 1500 bytes atraviesa 3 enlaces de 10 Mbps. Calcula el tiempo total ignorando propagación, procesamiento y colas.

Solución. $L = 1500 \cdot 8 = 12\,000$ bits. $d = N\,L/R = 3 \cdot \dfrac{12\,000}{10^7} = \mathbf{3{,}6\ ms}$.

Ejercicio 3: throughput

Un archivo de 4 MB (tomando 1 MB $= 8 \cdot 10^6$ bits) se transfiere por un camino con enlaces de 2 Mbps, 1 Mbps y 5 Mbps. ¿Cuánto tarda?

Solución. Throughput $= \min\{2, 1, 5\} = 1$ Mbps. $F = 32 \cdot 10^6$ bits $\Rightarrow T = \dfrac{32 \cdot 10^6}{10^6} = \mathbf{32\ s}$.

Ejercicio 4: intensidad de tráfico

Un router transmite por un enlace de 10 Mbps paquetes de 12 000 bits que llegan a 700 paquetes/s. ¿Cuál es la intensidad de tráfico y qué implica?

Solución. $\dfrac{La}{R} = \dfrac{12\,000 \cdot 700}{10^7} = \mathbf{0{,}84}$. Es menor que 1, así que las colas no crecen sin límite, pero el retardo de cola ya es notable; si a subiera por encima de ≈ 833 paquetes/s la intensidad superaría 1 y habría pérdidas.

Ejercicio 5: producto ancho de banda-retardo

Enlace de 100 Mbps con RTT $= 40$ ms (retardo de propagación de ida 20 ms). Calcula $R \cdot \text{RTT}$ y $R \cdot d_{\text{prop}}$.

Solución. $R \cdot \text{RTT} = 10^8 \cdot 0{,}04 = 4 \cdot 10^6$ bits (500 kB). $R \cdot d_{\text{prop}} = 10^8 \cdot 0{,}02 = 2 \cdot 10^6$ bits (250 kB).

Ejercicio 6: HTTP

Una página tiene un archivo base y 10 imágenes, en el mismo servidor. RTT $= 50$ ms; los tiempos de transmisión son despreciables. Calcula el tiempo de descarga en los tres modos.

Solución.

- No persistente (en serie): $22 \cdot 50 = \mathbf{1{,}1\ s}$.
- Persistente sin *pipelining*: $12 \cdot 50 = \mathbf{0{,}6\ s}$.
- Persistente con *pipelining*: $3 \cdot 50 = \mathbf{0{,}15\ s}$.

Ejercicio 7: multiplexación estadística

Un enlace de 3 Mbps lo comparten usuarios que, activos, transmiten a 150 kbps, y lo están el 10 % del tiempo. ¿Cuántos usuarios admite la conmutación de circuitos? Con 120 usuarios en conmutación de paquetes, ¿cuál es la probabilidad de que haya más usuarios activos que los que cabrían en un circuito?

Solución. Circuitos: $3\,\text{Mbps}/150\,\text{kbps} = \mathbf{20}$ usuarios. Paquetes: $N \sim \text{Bin}(120, 0{,}1)$ y se pide $P(N > 20) = \sum_{k=21}^{120} \binom{120}{k} 0{,}1^k 0{,}9^{120-k} \approx \mathbf{0{,}008}$ (menos del 1 %).

Ejercicio 8: cliente-servidor frente a P2P

Se distribuye un archivo de $F = 10$ Gbit a $N = 20$ pares. El servidor sube a $u_s = 20$ Mbps; cada par sube a $u = 1$ Mbps y baja a $d = 10$ Mbps. Calcula el tiempo mínimo en ambos modelos.

Solución. Cliente-servidor: $\max\{20 \cdot 10^{10}/(2 \cdot 10^7),\ 10^{10}/10^7\} = \max\{10\,000,\ 1000\} = \mathbf{10\,000\ s}$. P2P: $\max\{10^{10}/(2 \cdot 10^7),\ 1000,\ 20 \cdot 10^{10}/(2 \cdot 10^7 + 20 \cdot 10^6)\} = \max\{500,\ 1000,\ 5000\} = \mathbf{5000\ s}$.

Ejercicio 9: caché web

Una red institucional se conecta a Internet por un enlace de acceso de 15 Mbps. Los objetos miden de media 1 Mbit y se piden 15 por segundo. El retardo desde el router de salida hasta el origen es de 2 s de media. Se instala una caché con tasa de aciertos $h = 0{,}4$. Calcula la intensidad en el enlace de acceso y el retardo medio (se desprecia el retardo de la LAN y de la caché).

Solución. Sin caché, $La/R = 15 \cdot 10^6/15 \cdot 10^6 = 1$: enlace saturado. Con caché solo el 60 % de las peticiones sale: 9 objetos/s $\Rightarrow$ $La/R = \mathbf{0{,}6}$. Con $\Delta = L/R = 1/15$ s y cola M/M/1, $d_{\text{acceso}} = \Delta/(1 - \beta\Delta) = 0{,}0667/0{,}4 \approx 0{,}167$ s ($\beta = 9$/s). Retardo medio: $0{,}4 \cdot 0 + 0{,}6\,(2 + 0{,}167) \approx \mathbf{1{,}3\ s}$.

Ejercicio 10: DNS más HTTP

Un usuario escribe una URL. Para resolver el nombre se visitan dos servidores DNS con RTT de 3 ms y 20 ms. Después, la página (archivo base y 2 imágenes en el mismo servidor, tiempos de transmisión despreciables) se descarga con HTTP no persistente en serie y $\text{RTT}_{\text{HTTP}} = 40$ ms. Calcula el tiempo total.

Solución. DNS (solo una vez): $3 + 20 = 23$ ms. HTTP: 3 objetos $\times$ 2 RTT $= 3 \cdot 80 = 240$ ms. Total $= \mathbf{263\ ms}$.

4. Problemas de exámenes anteriores

Cómo leer esta sección

Las soluciones son mías y no son oficiales. Donde el enunciado deja algo abierto (definición de eficiencia, si hay establecimiento TCP, qué se cuenta en el retardo de los nodos), indico la hipótesis. Comprueba con las soluciones del profesor si las publica. Convención usada: la **MTU** es el máximo de SDU del enlace, como en los enunciados de 2023–24 (MTU 1500, TCP 1460), y 1 KB = 1024 B solo cuando el enunciado lo dice.

4.1 Retardo, propagación y conmutación de circuitos

PEI1 2021–22, cuestión 1b: llamada con 4 nodos

Se transfieren 64 kbit por un circuito de 64 kbps con 4 nodos. Retardo de los nodos intermedios: 100 ms; del nodo final (respuesta): 500 ms; propagación por enlace: 10 ms. Tiempo hasta que los datos llegan al destino.

Solución (hipótesis: los nodos intermedios tardan lo mismo en ambos sentidos). Hay 3 enlaces y 2 nodos intermedios.

- Petición de establecimiento: $3 \cdot 10 + 2 \cdot 100 = 230$ ms. El destino responde: 500 ms. Confirmación de vuelta: 230 ms. Total de establecimiento: **960** ms.
- Datos: $d_{\text{trans}} = 64\,000/64\,000 = 1$ s y propagación $3 \cdot 10 = 30$ ms: 1030 ms.
- Total: $960 + 1030 =$ **1990 ms**.

En el diagrama se dibujan cuatro líneas verticales (nodos), la petición descendiendo en diagonal con el retardo de cada nodo, la respuesta de vuelta y por último el volcado de datos.

EE 2021–22, cuestión 1b: longitud de trama

Enlace de $D = 3$ km, $v_p = 3 \cdot 10^8$ m/s y $R = 1$ Gbps. ¿Qué longitud debe tener la trama para que, cuando llegue su primer bit, salga el último?

Solución. $d_{\text{prop}} = 3000/(3 \cdot 10^8) = 10\ \mu$s. Se pide $L/R = d_{\text{prop}}$, así que $L = R\,d_{\text{prop}} = 10^9 \cdot 10^{-5} =$ **10 000 bits**.

PEI1 2023–24 (turno 1), problema 1: parada y espera

A y B están a 1000 km, $R = 20$ Mbps y $v_p = 2{,}5 \cdot 10^8$ m/s, luego $d_{\text{prop}} = 4$ ms.

a) Retardo extremo a extremo de 24 ms con envío continuo: $L/R + 4 = 24 \Rightarrow L/R = 20$ ms, es decir $L =$ **400 000 bits**.

b) 600 000 bits fragmentados en n paquetes, con parada y espera: la transmisión total es $600\,000/(20 \cdot 10^6) = 30$ ms y cada fragmento añade $2 \cdot 4 = 8$ ms de ida y vuelta. $30 + 8n = 94 \Rightarrow n =$ **8**.

c) Con un ACK cada 5 fragmentos y $T = 36$ ms, el planteamiento $30 + 8 \cdot (\text{bloques}) = 36$ no da un número entero de bloques, así que el enunciado parece tener una errata y no resuelvo este apartado. El mismo apartado en el turno 2 (abajo) sí es coherente con 36 ms.

PEI1 2023–24 (turno 2), problema 1

Mismo enlace ($R = 20$ Mbps, $d_{\text{prop}} = 4$ ms), fichero de 400 000 bits.

a) Envío continuo: $400\,000/20 \cdot 10^6 + 4 = 20 + 4 =$ **24** ms (hasta recibirlo; el ACK despreciable añade otros 4 ms si se pide la confirmación).

b) 10 paquetes de 40 000 bits (2 ms cada uno), parada y espera: $10\,(2+4+4) =$ **100 ms**.

c) ACK cada 5 paquetes y el emisor espera cada ACK: 2 bloques de $5 \cdot 2 = 10$ ms de transmisión: $2\,(10+4+4) =$ **36 ms**.

4.2 Encapsulación, MTU, eficiencia y errores

PEI1 2022–23, cuestión 3

Fichero de aplicación de 2040 B entre A y B a 70 m, $v_p = 2{,}5 \cdot 10^8$ m/s, $R = 256$ kbps, MTU 100 B; cabeceras de enlace 28 B, de red 20 B y de transporte 20 B.

a) SDU de enlace = 100 B (la MTU). SDU de red = $100 - 20 = 80$ B (ahí va el segmento). SDU de transporte = $80 - 20 = 60$ B de datos. Fragmentos: $2040/60 = 34$. Cada trama mide $60 + 20 + 20 + 28 = 128$ B $= 1024$ bits, así que $d_{\text{trans}} = 1024/256\,000 = 4$ ms. Sin confirmación: $34 \cdot 4$ ms $+ d_{\text{prop}} =$ **136 ms** ($d_{\text{prop}} = 70/(2{,}5 \cdot 10^8) = 0{,}28$ μs, despreciable). Caudal a nivel de transporte (datos de aplicación entregados): $2040 \cdot 8/0{,}136 =$ **120 kbps**.

b) ACK cada 10 fragmentos y el último siempre confirmado: $34 = 10 + 10 + 10 + 4$, es decir 4 ACK. Cada ACK lleva 60 B de datos y mide también 128 B, luego 4 ms. El emisor espera cada ACK: $136 + 4\,(4 + 2 \cdot 0{,}00028) \approx$ **152,002 ms**.

c) BER $= 10^{-4}$ y $L = 1024$ bits: $P_{\text{error}} = 1 - (1 - 10^{-4})^{1024} \approx 0{,}0973$. Tramas por segundo: $256\,000/1024 = 250$. Erróneas por segundo: $250 \cdot 0{,}0973 \approx$ **24,3**.

PEI1 2023–24 (turno 1), problema 2

Enlace punto a punto de 14,8904 Mbps y 2,5 km ($v_p = 280\,000$ km/s). Mensajes de 528 B con cabeceras de enlace 16 B, red 20 B y transporte 15 B.

Con la definición de la asignatura, $E_N = \text{SDU}_N/\text{TBCF}$, la eficiencia de la **capa de red** usa como SDU_{red} el segmento entero: $528 + 15 = 543$ B. La trama completa mide TBCF $= 528 + 15 + 20 + 16 = 579$ B.

a) $E_{\text{red}} = \dfrac{543}{579} =$ **93,8** %. (La de la capa de transporte o de aplicación, con solo los 528 B de datos, sería 91,2 %.)

b) Trama de $579 \cdot 8 = 4632$ bits y BER $= 10^{-5}$: $P_{ET} = 1 - (1 - 10^{-5})^{4632} \approx 0{,}0453$. $E = \dfrac{543}{579\,(1 + 0{,}0453)} \approx$ **89,7** %. Con el valor exacto (reintentos hasta acertar) saldría $0{,}9378 \cdot 0{,}9547 \approx 89{,}5$ %.

c) Sin errores y con $T = 320$ μs (microsegundos, no segundos). Trama completa: 4632 bits, con $d_{\text{trans}} = 4632/(14{,}8904 \cdot 10^6) = 311{,}07$ μs y $d_{\text{prop}} = 2{,}5/280\,000 = 8{,}93$ μs. Su suma es **320 ¯s** exactos, así que se transmite **una sola trama** y el fichero mide **528 B** (los datos de un mensaje). Que cuadre al microsegundo confirma que la trama incluye las tres cabeceras (579 B).

PEI1 2023–24 (turno 2), problema 2: FastEthernet

Cliente y servidor a 250 km, $v_p = 2{,}5 \cdot 10^8$ m/s ($d_{\text{prop}} = 1$ ms), $R = 100$ Mbps. Se envían 2920 B. Enlace: MTU 1500, PCI de $22 + 4$ B; IP: PCI 20 B; TCP: PCI 20 B. El servidor devuelve un ACK con PDU de transporte de 20 B.

a) SDU de TCP $= 1500 - 20 - 20 = 1460$ B, así que $2920/1460 = 2$ segmentos TCP de 1480 B, 2

datagramas IP de 1500 B y 2 tramas de 1526 B (PCI de enlace de 26 B). El servidor genera 1 ACK: segmento de 20 B, datagrama de 40 B y trama de 66 B.

b) $d_{\text{trans}}(\text{dato}) = 1526 \cdot 8/10^8 = 0{,}12208$ ms; $d_{\text{trans}}(\text{ACK}) = 66 \cdot 8/10^8 = 0{,}00528$ ms. El cliente envía las dos tramas seguidas (0,24416 ms). La última llega al servidor en $0{,}24416 + 1 = 1{,}24416$ ms. El servidor responde y el ACK llega a: $1{,}24416 + 0{,}00528 + 1 =$ **2,24944 ms**.

c) Caudal de aplicación, sin enviar un archivo nuevo hasta recibir el ACK: $2920 \cdot 8/2{,}24944 \cdot 10^{-3} \approx$ **10,38 Mbps**.

EE 2023–24, ejercicio 1

Cliente envía 2920 B de aplicación. El servidor devuelve de inmediato un ACK con SDU de transporte de 701 B por cada bloque de 1460 B recibido. $R = 1{,}2144$ Mbps, 250 km ($d_{\text{prop}} = 1$ ms), MTU 1500 con PCI de enlace de 18 B, IP 20 B, TCP 20 B.

a) Cliente: 2 PDU de transporte, cada una de 1460 datos + 20 control (segmento de 1480 B), que bajan a datagramas de 1500 B y tramas de 1518 B. Servidor: 2 ACK, cada uno de 701 datos + 20 control, es decir tramas de $701 + 20 + 20 + 18 = 759$ B.

b) $d_{\text{trans}}(\text{datos}) = 1518 \cdot 8/1{,}2144 \cdot 10^6 =$ **10 ms**; $d_{\text{trans}}(\text{ACK}) = 759 \cdot 8/1{,}2144 \cdot 10^6 =$ **5 ms**; $d_{\text{prop}} = 250\,000/2{,}5 \cdot 10^8 =$ **1 ms**.

c1) El cliente envía las dos tramas seguidas (20 ms). El primer ACK sale cuando el servidor recibe la primera trama ($10 + 1 = 11$ ms). El segundo ACK, al recibir la segunda ($20 + 1 = 21$ ms), ocupa el enlace de 21 a 26 ms y llega al cliente a $26 + 1 =$ **27 ms**, el retardo extremo a extremo.

c2) Con propagación d: el servidor recibe todos los datos en $20 + d$. El primer ACK sale a $10 + d$, tarda 5 ms en transmitirse y llega a $15 + 2d$. Igualando $20 + d = 15 + 2d$ se obtiene $d =$ **5 ms**.

EE 2022–23, cuestión 5: Alicia y Bernardo

Enlace punto a punto de 200 km, $v_p = 2{,}5 \cdot 10^8$ m/s ($d_{\text{prop}} = 0{,}8$ ms), $R = 10$ Mbps, full dúplex, sin errores. 1 KB = 1024 B.

a) 6 KB = 49 152 bits $\Rightarrow d_{\text{trans}} = 4{,}9152$ ms. Envío continuo y ACK despreciable: $4{,}9152 + 2 \cdot 0{,}8 =$ **6,5152 ms**.

b) 3 fragmentos de 1 KB (8192 bits, 0,8192 ms), parada y espera: $3\,(0{,}8192 + 1{,}6) =$ **7,2576 ms**.

c) 8 fragmentos, ACK cada 2 y el emisor no se bloquea (full dúplex): envía $8 \cdot 0{,}8192 = 6{,}5536$ ms seguidos y el último ACK sale al recibir el último fragmento: $6{,}5536 + 1{,}6 =$ **8,1536 ms**.

d) Caudal = bits de aplicación / tiempo: a) $49\,152/6{,}5152$ ms $\approx$ **7,54 Mbps**; b) $24\,576/7{,}2576$ ms $\approx$ **3,39 Mbps**; c) $65\,536/8{,}1536$ ms $\approx$ **8,04 Mbps**. El mejor es c), porque aprovecha el dúplex y no se bloquea; el peor, b), por la parada y espera.

4.3 Tiempos HTTP

PP2 turno 1 2023–24: HTTP/1.1 con y sin *pipelining*

$R = 400$ kbps, $d_{\text{prop}} = 2{,}5$ ms, MTU 32 000 B, cabeceras de enlace 12, red 20 y transporte 20 B (52 B por trama). GET: 2448 B; `peliculas.html`: 1948 B; `logo.jpg`: 1448 B; `fondo.jpg`: 3948 B. Se ignora TCP.

Tramas: GET 2500 B = 20 000 bits $\Rightarrow$ 50 ms; HTML 2000 B $\Rightarrow$ 40 ms; logo 1500 B $\Rightarrow$ 30 ms; fondo 4000 B $\Rightarrow$ 80 ms.

a) Sin *pipelining*. Cada objeto: $50 + 2{,}5 + d_{\text{resp}} + 2{,}5$. HTML: 95 ms; logo: 85 ms; fondo: 135 ms.

Total = **315 ms**.

b) Con *pipelining*. HTML recibido a 95 ms. El cliente envía los dos GET seguidos (50 + 50 ms). El servidor recibe el primero a 95 + 50 + 2,5 = 147,5 y responde con el logo (30 ms): llega a 147,5 + 30 + 2,5 = 180 ms. Recibe el segundo GET a 95 + 100 + 2,5 = 197,5 ms (el servidor ya está libre) y responde con el fondo (80 ms): llega a 197,5 + 80 + 2,5 = **280 ms**.

PP1r 2023–24, ejercicio 2d: dos servidores y caché

Cliente, servidor A (no permite caché) y servidor B (sí). $R = 160$ kbps, $d_{\text{prop}} = 10$ ms, cabeceras 10 + 20 + 20 = 50 B. GET: 450 B; `200 OK`: 250 B + contenido; `304`: 350 B; HTML: 1200 B; imagen: 1700 B. HTTP/1.1 sin *pipelining*; el cliente carga la página y la refresca.

Tramas: GET 500 B = 25 ms; `200 OK` con HTML 1500 B = 75 ms; `200 OK` con imagen 2000 B = 100 ms; `304` 400 B = 20 ms.

- Primera carga: HTML en A: 25 + 10 + 75 + 10 = 120 ms; imagen en B: 25 + 10 + 100 + 10 = 145 ms.
- Refresco: el HTML (sin caché) vuelve a pedirse a A: 120 ms. La imagen está en caché y no ha cambiado: GET condicional a B con `304`: 25 + 10 + 20 + 10 = 65 ms.
- Total: 120 + 145 + 120 + 65 = **450 ms**.

PP2 junio 2023, preguntas c a f: interacción HTTP

Petición `GET /peliculas/humor.html HTTP/1.1` con `If-Modified-Since: ___`. Respuesta `200 OK` con `Last-Modified: Mon, 23 Oct 2023 05:59:01 GMT` y cuerpo.

- **c)** Método: `GET`.
- **d)** Código de estado: `200 OK`, petición correcta y el objeto va en el cuerpo.
- **e)** Cualquier fecha **anterior** a `Mon, 23 Oct 2023 05:59:01 GMT`: el objeto es más nuevo que la copia del cliente, así que el servidor lo devuelve. También valdría omitir la cabecera, pero el enunciado pide un valor.
- **f)** Si la fecha fuera **igual o posterior** a `Last-Modified`, el servidor respondería `304 Not Modified`, sin cuerpo, y el cliente usaría su copia en caché.

PP2 turno 2 2023–24, apartados d y e: web con imagen, vídeo y enlace

`index.html` (1300 B), `Calendario.png` (2600 B; modificado el 19/10/2023 10:30:30 y se actualiza cada 10 min), `Demostracion.avi` (5200 B; modificado el 21/9/2022 20:45:56) y un enlace a `apuntate.html`.

d) Se descargan `index.html` y los objetos incrustados (PNG y AVI). `apuntate.html` es un hiperenlace: según tus apuntes no se pide hasta que el usuario pulsa. Como el enunciado da su tamaño, confírmalo con el profesor. MTU 1440: el segmento de TCP lleva como máximo 1440 − 30 − 60 = 1350 B, y cada mensaje HTTP suma 50 B de cabecera.

e) 30 minutos después, el cliente hace GET condicional con el `Last-Modified` que guardó. El PNG se actualiza cada 10 minutos, así que ha cambiado: `200 OK` con el objeto. El AVI es de 2022 y no ha cambiado: `304 Not Modified`, sin cuerpo.

4.4 Cuestiones teóricas que se repiten

Cuestión	Respuesta corta
Servicios que puede dar el transporte	Transferencia fiable de datos, control de flujo, control de congestión, multiplexación y demultiplexación de aplicaciones (puertos), garantías de tasa o de retardo y seguridad.
Qué ofrecen TCP y UDP, y qué no	TCP: orientado a conexión, fiable, en orden, con control de flujo y de congestión. UDP: solo multiplexación y detección de errores (suma de comprobación), sin conexión ni fiabilidad. **Ninguno** garantiza tasa mínima ni retardo máximo, y ninguno incluye seguridad (la aporta TLS en la aplicación).
Conmutación de circuitos: ventajas e inconvenientes	Ventajas: retardo constante y bajo, caudal garantizado, sin congestión una vez establecido. Inconvenientes: establecimiento previo, recursos desaprovechados en los silencios, baja eficiencia con tráfico en ráfagas, bloqueo si no hay circuitos libres.
Cuatro diferencias, conectivo frente a no conectivo	Estado en los nodos (sí o no), establecimiento previo (retrasa o no el inicio), orden de entrega (garantizado o no), procesamiento por nodo (menor o mayor) y tolerancia a fallos (peor o mejor).
Tres métodos HTTP	`GET` pide un objeto; `POST` envía datos en el cuerpo; `HEAD` como GET sin cuerpo; `PUT` sube un objeto; `DELETE` lo borra.
HTTP: ¿con o sin estado?	Sin estado. Para identificar al usuario o recordar sus preferencias se usan **cookies**.

4.5 Test: respuestas razonadas

PEI1 2023–24, turno 1: 1 b; 2 d; 3 b; 4 b; 5 d (a, b y c son ciertas); 6 b; 7 d (el servicio conectivo exige estado, garantiza el orden y tiene menos procesamiento por nodo); 8 d; 10 b. La 9 es ambigua: "retardos no constantes" y "puede haber congestión" son ciertas en un circuito virtual, y la opción "todas incorrectas" sería falsa.

PEI1 2023–24, turno 2: 1 g (el modelo cliente-servidor tiene un servidor centralizado; las opciones van rotuladas e a h); 2 b; 3 c; 4 a; 5 d (el servicio no conectivo no garantiza la secuencia); 6 d (IP no implementa control de errores ni de flujo); 7 b; 8 b; 9 c (el establecimiento previo es la desventaja); 10 d (el retardo que describe es el de transmisión, L/R, y no figura entre las opciones).

5. Hoja de fórmulas

Concepto	Fórmula
Retardo de transmisión	$d_{\text{trans}} = L/R$
Retardo de propagación	$d_{\text{prop}} = d/s$
Retardo nodal	$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{cola}} + d_{\text{trans}} + d_{\text{prop}}$
Retardo extremo a extremo (N enlaces)	$N\,(d_{\text{proc}} + d_{\text{trans}} + d_{\text{prop}})$
Store-and-forward puro	$N\,L/R$
Intensidad de tráfico	La/R
Throughput	$\min\{R_1, \ldots, R_N\}$; $T = F/\text{throughput}$
Producto ancho de banda-retardo	$R \cdot d_{\text{prop}}$ (enlace) o $R \cdot \text{RTT}$ (en vuelo, transporte)
HTTP no persistente (un objeto)	$2\,\text{RTT} + t_{\text{trans}}$
Retardo medio con caché	$h\,d_{\text{caché}} + (1-h)\,d_{\text{origen}}$
Eficiencia y caudal	$E_N = \text{SDU}_N/\text{TBCF}$; $C_N = \text{SDU}_N/T_N$
Error de trama con BER	$P_{ET} = 1-(1-\text{BER})^L$; tramas totales $\approx$ tramas $(1+P_{ET})$; erróneas/s $= \frac{R}{L}P_{ET}$
Longitud para llenar el enlace	$L = R \cdot d_{\text{prop}}$
Parada y espera (n fragmentos)	$n\,(L/R + 2\,d_{\text{prop}})$
Bloques de k con espera	$\lceil n/k \rceil\,(kL/R + 2\,d_{\text{prop}})$
SDU máxima	$\text{MTU} - \text{PCI}_{\text{red}} - \text{PCI}_{\text{transp}}$
Distribución cliente-servidor	$\max\{NF/u_s,\ F/d_{\min}\}$
Distribución P2P	$\max\{F/u_s,\ F/d_{\min},\ NF/(u_s + \sum u_i)\}$
HTTP + DNS (no persistente, n objetos)	$\sum \text{RTT}_{\text{DNS}} + 2n\,\text{RTT}_{\text{HTTP}}$

Conversiones útiles. 1 byte = 8 bits; 1 kbps = 10^3 bps; 1 Mbps = 10^6 bps; 1 Gbps = 10^9 bps. En velocidades de transmisión se usan *potencias de 10*, no de 2.

Puertos y códigos que hay que saber

HTTP 80, HTTPS 443, DNS 53, SSH 22, SMTP 25. Estados HTTP: 200, 301, 304, 400, 404, 418, 500, 503, 505. Registros DNS: A, AAAA, NS, CNAME, MX.

6. Correcciones respecto al resumen original

Qué se ha corregido o aclarado

1. **FTTH**: el original decía que garantiza tasas *simétricas*. No siempre: GPON es asimétrica; XGS-PON sí es simétrica.
2. **Intensidad de tráfico**: se decía que el retardo crece de forma "exponencial y asintótica". Lo correcto: crece *rápidamente* y sin límite cuando $La/R \to 1$ (cola infinita).
3. **Producto ancho de banda-retardo**: el original solo daba $R \cdot \text{RTT}$. En Kurose & Ross el del cap. 1 es $R \cdot d_{\text{prop}}$; ahora se explican ambos y cuándo usar cada uno.

4. **Transporte**: en TCP, el temporizador es el *RTO* (*retransmission timeout*); el original lo etiquetaba como "RTT estimado".
5. **PDU de UDP**: se aclara que Kurose & Ross usa "segmento" también para UDP.
6. **HTTP persistente**: se precisa que HTTP/1.1 usa conexiones persistentes por defecto y que el *pipelining* es opcional; se añade el cálculo de RTT para los tres modos.
7. **DNS iterativo/recursivo**: el original etiquetaba la iterativa como "recomendada". En la práctica, host $\rightarrow$ DNS local es recursiva y el DNS local $\rightarrow$ jerarquía suele ser iterativa.
8. **Erratas de texto** corregidas: "Arquitectura ópticas pasivas", "solicítan", "ancho de meganavegación" (debía ser *ancho de banda*), "Invación bloqueante", "adicionándole", "Retransmission Timeout − RTT estimado".
9. **Añadidos** que faltaban respecto al temario: medios físicos, clasificación de redes (topología, alcance, posición), estructura de Internet, organismos de normalización y regulación (RA3), *throughput*, formato de los mensajes HTTP, mensajes DNS, tabla de aplicaciones y transporte, programación con sockets en Python, ejercicios resueltos y hoja de fórmulas.
10. **Ampliaciones finales**: tabla de correspondencia con la guía docente, tiempo de distribución cliente-servidor frente a P2P (Kurose & Ross, cap. 2) y ejercicios 8–10 (P2P, caché, DNS + HTTP).
11. **Segunda ampliación** (a partir de exámenes de cursos anteriores): código `418`, red del hogar, PDU/SDU/PCI, servicios conectivos y circuitos virtuales, método de encapsulación con MTU, eficiencia, BER y confirmaciones, ejemplos de DNS y de HTTP con caché, y la sección de problemas resueltos.
12. **Tercera revisión**: la eficiencia y el caudal pasan a la definición de la asignatura ($E_N = \text{SDU}_N/\text{TBCF}$; la de capa de red del PP1 2023–24 pasa de 91,2 % a 93,8 %), se corrige el tiempo del problema 2c a 320 μs, y se añaden FTP, correo, BitTorrent, el formato de la prueba y el apartado e del PP2 turno 2.